



Red Hat OpenShift AI Self-Managed 2.25

Deploying models

Deploy models in Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 2.25 Deploying models

Deploy models in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

As a Red Hat OpenShift AI user, you can deploy your machine-learning models in Red Hat OpenShift AI Self-Managed.

Table of Contents

CHAPTER 1. STORING MODELS	3
1.1. USING OCI CONTAINERS FOR MODEL STORAGE	3
1.2. STORING A MODEL IN AN OCI IMAGE	3
1.3. UPLOADING MODEL FILES TO A PERSISTENT VOLUME CLAIM (PVC)	5
CHAPTER 2. DEPLOYING MODELS ON THE SINGLE-MODEL SERVING PLATFORM	7
2.1. ABOUT KSERVE DEPLOYMENT MODES	7
2.2. DEPLOYING MODELS ON THE SINGLE-MODEL SERVING PLATFORM	8
2.3. DEPLOYING A MODEL STORED IN AN OCI IMAGE BY USING THE CLI	12
2.4. DEPLOYING MODELS BY USING DISTRIBUTED INFERENCE WITH LLM-D	15
2.4.1. Example usage for Distributed Inference with llm-d	17
2.4.1.1. Single-node GPU deployment	17
2.4.1.2. Multi-node deployment	18
2.4.1.3. Intelligent inference scheduler with KV cache routing	18
2.5. MONITORING MODELS ON THE SINGLE-MODEL SERVING PLATFORM	18
2.5.1. Viewing performance metrics for a deployed model	18
2.5.2. Viewing model-serving runtime metrics for the single-model serving platform	19
CHAPTER 3. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM	22
3.1. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM	22
3.2. VIEWING NVIDIA NIM METRICS FOR A NIM MODEL	24
3.3. VIEWING PERFORMANCE METRICS FOR A NIM MODEL	25
CHAPTER 4. DEPLOYING MODELS ON THE MULTI-MODEL SERVING PLATFORM	27
4.1. ADDING A MODEL SERVER FOR THE MULTI-MODEL SERVING PLATFORM	27
4.2. DELETING A MODEL SERVER	29
4.3. DEPLOYING A MODEL BY USING THE MULTI-MODEL SERVING PLATFORM	30
4.4. VIEWING A DEPLOYED MODEL	31
4.5. UPDATING THE DEPLOYMENT PROPERTIES OF A DEPLOYED MODEL	32
4.6. DELETING A DEPLOYED MODEL	33
4.7. CONFIGURING MONITORING FOR THE MULTI-MODEL SERVING PLATFORM	33
4.8. VIEWING MODEL-SERVING RUNTIME METRICS FOR THE MULTI-MODEL SERVING PLATFORM	34
4.9. VIEWING PERFORMANCE METRICS FOR ALL MODELS ON A MODEL SERVER	35
4.10. VIEWING HTTP REQUEST METRICS FOR A DEPLOYED MODEL	36
CHAPTER 5. MAKING INFERENCE REQUESTS TO DEPLOYED MODELS	38
5.1. ACCESSING THE AUTHENTICATION TOKEN FOR A DEPLOYED MODEL	38
5.2. ACCESSING THE INFERENCE ENDPOINT FOR A DEPLOYED MODEL	38
5.3. MAKING INFERENCE REQUESTS TO MODELS DEPLOYED ON THE SINGLE-MODEL SERVING PLATFORM	39
5.4. INFERENCE ENDPOINTS	39
5.4.1. Caikit TGIS ServingRuntime for KServe	39
5.4.2. Caikit Standalone ServingRuntime for KServe	40
5.4.3. TGIS Standalone ServingRuntime for KServe	41
5.4.4. OpenVINO Model Server	41
5.4.5. vLLM NVIDIA GPU ServingRuntime for KServe	41
5.4.6. vLLM Intel Gaudi Accelerator ServingRuntime for KServe	42
5.4.7. vLLM AMD GPU ServingRuntime for KServe	42
5.4.8. vLLM Spyre AI Accelerator ServingRuntime for KServe	42
5.4.9. NVIDIA Triton Inference Server	43
5.4.10. Seldon MLServer	44
5.4.11. Additional resources	45

CHAPTER 1. STORING MODELS

You must store your model before you can deploy it. You can store a model in an S3 bucket, URI or Open Container Initiative (OCI) containers.

1.1. USING OCI CONTAINERS FOR MODEL STORAGE

As an alternative to storing a model in an S3 bucket or URI, you can upload models to Open Container Initiative (OCI) containers. Deploying models from OCI containers is also known as modelcars in KServe.

Using OCI containers for model storage can help you:

- Reduce startup times by avoiding downloading the same model multiple times.
- Reduce disk space usage by reducing the number of models downloaded locally.
- Improve model performance by allowing pre-fetched images.

Using OCI containers for model storage involves the following tasks:

- Storing a model in an OCI image.
- Deploying a model from an OCI image by using either the user interface or the command line interface. To deploy a model by using:
 - The user interface, see [Deploying models on the single-model serving platform](#).
 - The command line interface, see [Deploying a model stored in an OCI image by using the CLI](#).

1.2. STORING A MODEL IN AN OCI IMAGE

You can store a model in an OCI image. The following procedure uses the example of storing a MobileNet v2-7 model in ONNX format.

Prerequisites

- You have a model in the ONNX format. The example in this procedure uses the MobileNet v2-7 model in ONNX format.
- You have installed the Podman tool.

Procedure

1. In a terminal window on your local machine, create a temporary directory for storing both the model and the support files that you need to create the OCI image:

```
cd $(mktemp -d)
```

2. Create a **models** folder inside the temporary directory:

```
mkdir -p models/1
```

**NOTE**

This example command specifies the subdirectory **1** because OpenVINO requires numbered subdirectories for model versioning. If you are not using OpenVINO, you do not need to create the **1** subdirectory to use OCI container images.

- Download the model and support files:

```
DOWNLOAD_URL=https://github.com/onnx/models/raw/main/validated/vision/classification/mobilenet/model/mobilenetv2-7.onnx
curl -L $DOWNLOAD_URL -O --output-dir models/1/
```

- Use the **tree** command to confirm that the model files are located in the directory structure as expected:

```
tree
```

The **tree** command should return a directory structure similar to the following example:

```
.
├── Containerfile
├── models
│   └── 1
│       └── mobilenetv2-7.onnx
```

- Create a Docker file named **Containerfile**:

**NOTE**

- Specify a base image that provides a shell. In the following example, **ubi9-micro** is the base container image. You cannot specify an empty image that does not provide a shell, such as **scratch**, because KServe uses the shell to ensure the model files are accessible to the model server.
- Change the ownership of the copied model files and grant read permissions to the root group to ensure that the model server can access the files. OpenShift runs containers with a random user ID and the root group ID.

```
FROM registry.access.redhat.com/ubi9/ubi-micro:latest
COPY --chown=0:0 models /models
RUN chmod -R a=rX /models

# nobody user
USER 65534
```

- Use **podman build** commands to create the OCI container image and upload it to a registry. The following commands use Quay as the registry.

**NOTE**

If your repository is private, ensure that you are authenticated to the registry before uploading your container image.


```
podman build --format=oci -t quay.io/<user_name>/<repository_name>:<tag_name> .
podman push quay.io/<user_name>/<repository_name>:<tag_name>
```

1.3. UPLOADING MODEL FILES TO A PERSISTENT VOLUME CLAIM (PVC)

When deploying a model, you can serve it from a preexisting Persistent Volume Claim (PVC) where your model files are stored. You can upload your local model files to a PVC in the IDE that you access from a running workbench.

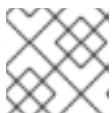
Prerequisites

- You have access to the OpenShift AI dashboard.
- You have access to a project that has a running workbench.
- You have created a persistent volume claim (PVC) with a context type of **Model storage**.
- The workbench is attached to the persistent volume (PVC).
 - For instructions on attaching a PVC, see [Creating a project workbench](#).
- You have the model files saved on your local machine.

Procedure

Follow these steps to upload your model files to the PVC mount point (**/opt/app-root/src/**) within your workbench:

1. From the OpenShift AI dashboard, click the open icon () to open your IDE in a new window.
2. In your IDE, navigate to the **File Browser** pane on the left-hand side.
 - a. In JupyterLab, this is usually labeled **Files**.
 - b. In code-server, this is usually the **Explorer** view.
3. In the file browser, navigate to the **/opt/app-root/src/** folder. This folder represents the root of your attached PVC.



NOTE

Any files or folders that you create or upload to this folder persist in the PVC.

4. Optional: Create a new folder to organize your models:
 - a. In the file browser, right-click within the **/opt/app-root/src/** folder in the file browser and select **New Folder**.
 - b. Name the folder (for example, **models**).
 - c. Double-click the new **models** folder to enter it.
5. Upload your model files to the current folder (**/opt/app-root/src/** or **/opt/app-root/src/models/**):

- **Using JupyterLab:**

- a. Click the **Upload Files** icon () in the file browser toolbar above the folder listing.
- b. In the file selection dialog, navigate to and select the model files from your local computer. Click **Open**.
- c. Wait for the upload progress bars next to the filenames to complete.

- **Using code-server:**

- a. Drag the model files directly from your local file explorer and drop them into the file browser pane in the target folder within code-server.
6. Wait for the upload process to complete.

Verification

- Confirm that your files appear in the file browser at the path where you uploaded them.

Next steps

When you follow the procedure to deploy a model, you can access the model files from the specified path within your PVC:

1. In the **Deploy model** dialog, select **Existing cluster storage** under the **Source model location** section.
2. From the **Cluster storage** list, select the PVC associated with your workbench.
3. In the **Model path** field, enter the path to your model or the folder containing your model.

CHAPTER 2. DEPLOYING MODELS ON THE SINGLE-MODEL SERVING PLATFORM

The single-model serving platform deploys each model from its own dedicated model server. This architecture is ideal for deploying, monitoring, scaling, and maintaining large models that require more resources, such as large language models (LLMs).

The platform is based on the KServe component and offers two deployment modes:

- **KServe RawDeployment:** Uses a standard deployment method that does not require serverless dependencies.
- **Knative Serverless:** Uses Red Hat OpenShift Serverless for deployments that can automatically scale based on demand.

2.1. ABOUT KSERVE DEPLOYMENT MODES

KServe offers two deployment modes for serving models. The default mode, Knative Serverless, is based on the open-source Knative project and provides powerful autoscaling capabilities. It integrates with Red Hat OpenShift Serverless and Red Hat OpenShift Service Mesh. Alternatively, the KServe RawDeployment mode offers a more traditional deployment method with fewer dependencies.

Before you choose an option, understand how your initial configuration affects future deployments:

- **If you configure for Knative Serverless** You can use both Knative Serverless and KServe RawDeployment modes.
- **If you configure for KServe RawDeployment only.** You can only use the KServe RawDeployment mode.

Use the following comparison to choose the option that best fits your requirements.

Table 2.1. Comparison of deployment modes

Criterion	Knative Serverless	KServe RawDeployment
Default mode	Yes	No
Recommended use case	Most workloads.	Custom serving setups or models that must remain active.
Autoscaling	• Scales up automatically based on request volume. • Supports scaling down to zero when idle to save costs.	• No built-in autoscaling; you can configure Kubernetes Event-Driven Autoscaling (KEDA) or Horizontal Pod Autoscaler (HPA) on your deployment. • Does not support scaling to zero by default, which might result in higher costs during periods of low traffic.

Criterion	Knative Serverless	KServe RawDeployment
Dependencies	- Red Hat OpenShift Serverless Operator - Red Hat OpenShift Service Mesh - Authorino. Only required only if you enable token authentication and external routes.	None; uses standard Kubernetes resources such as Deployment , Service , and Horizontal Pod Autoscaler .
Configuration flexibility	Has some customization limitations inherited from Knative compared to raw Kubernetes deployments.	Provides full control over pod specifications because it uses standard Kubernetes Deployment resources.
Resource footprint	Larger, due to the additional dependencies required for serverless functionality.	Smaller.
Setup complexity	Might require additional configuration in setup and management. If Serverless is not already installed on the cluster, you must install and configure it.	Requires a simpler setup with fewer dependencies.

2.2. DEPLOYING MODELS ON THE SINGLE-MODEL SERVING PLATFORM

When you have enabled the single-model serving platform, you can enable a preinstalled or custom model-serving runtime and deploy models on the platform.

You can use preinstalled model-serving runtimes to start serving models without modifying or defining the runtime yourself. For help adding a custom runtime, see [Adding a custom model-serving runtime for the single-model serving platform](#).

To successfully deploy a model, you must meet the following prerequisites.

General prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have installed KServe and enabled the single-model serving platform.
- (Knative Serverless deployments only) To enable token authentication and external model routes for deployed models, you have added Authorino as an authorization provider. For more information, see [Adding an authorization provider for the single-model serving platform](#).
- (Knative Serverless deployments only) To enable token authentication and external model routes for deployed models, you have added Authorino as an authorization provider.
- You have created a data science project.
- You have access to S3-compatible object storage, a URI-based repository, an OCI-compliant

registry or a persistent volume claim (PVC) and have added a connection to your data science project. For more information about adding a connection, see [Adding a connection to your data science project](#).

- If you want to use graphics processing units (GPUs) with your model server, you have enabled GPU support in OpenShift AI. If you use NVIDIA GPUs, see [Enabling NVIDIA GPUs](#). If you use AMD GPUs, see [AMD GPU integration](#).

Runtime-specific prerequisites

Meet the requirements for the specific runtime you intend to use.

- **Caikit-TGIS runtime**
 - To use the Caikit-TGIS runtime, you have converted your model to Caikit format. For an example, see [Converting Hugging Face Hub models to Caikit format](#) in the [caikit-tgis-serving](#) repository.
- **vLLM NVIDIA GPU ServingRuntime for KServe**
 - To use the **vLLM NVIDIA GPU ServingRuntime for KServer**runtime, you have enabled GPU support in OpenShift AI and have installed and configured the Node Feature Discovery Operator on your cluster. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- **vLLM CPU ServingRuntime for KServe**
 - To use the VLLM runtime on IBM Z and IBM Power, use the **vLLM CPU ServingRuntime for KServe**. You cannot use GPU accelerators with IBM Z and IBM Power architectures. For more information, see [Red Hat OpenShift Multi Architecture Component Availability Matrix](#).
- **vLLM Intel Gaudi Accelerator ServingRuntime for KServe**
 - To use the **vLLM Intel Gaudi Accelerator ServingRuntime for KServer**runtime, you have enabled support for hybrid processing units (HPUs) in OpenShift AI. This includes installing the Intel Gaudi Base Operator and configuring a hardware profile. For more information, see [Intel Gaudi Base Operator OpenShift installation](#) in the AMD documentation and [Working with hardware profiles](#).
- **vLLM AMD GPU ServingRuntime for KServe**
 - To use the **vLLM AMD GPU ServingRuntime for KServer**runtime, you have enabled support for AMD graphic processing units (GPUs) in OpenShift AI. This includes installing the AMD GPU operator and configuring a hardware profile. For more information, see [Deploying the AMD GPU operator on OpenShift](#) and [Working with hardware profiles](#).
- **vLLM Spyre AI Accelerator ServingRuntime for KServe**



IMPORTANT

Support for IBM Spyre AI Accelerators on x86 is currently available in Red Hat OpenShift AI 2.25 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

- To use the **vLLM Spyre AI Accelerator ServingRuntime for KServer** runtime on x86, you have installed the Spyre Operator and configured a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).

Procedure

1. In the left menu, click **Data science projects**.
2. Click the name of the project that you want to deploy a model in.
A project details page opens.
3. Click the **Models** tab.
4. Click **Select single-model** to deploy your model using single-model serving.
5. Click the **Deploy model** button.
The **Deploy model** dialog opens.
6. In the **Model deployment name** field, enter a unique name for the model that you are deploying.
7. In the **Serving runtime** field, select an enabled runtime. If project-scoped runtimes exist, the **Serving runtime** list includes subheadings to distinguish between global runtimes and project-scoped runtimes.
8. From the **Model framework (name - version)** list, select a value if applicable.
9. From the **Deployment mode** list, select **KServe RawDeployment** or **Knative Serverless**. For more information about deployment modes, see [About KServe deployment modes](#).
10. In the **Number of model server replicas to deploy** field, specify a value.
11. The following options are only available if you have created a hardware profile:
 - a. From the **Hardware profile** list, select a hardware profile. If project-scoped hardware profiles exist, the **Hardware profile** list includes subheadings to distinguish between global hardware profiles and project-scoped hardware profiles.



IMPORTANT

By default, hardware profiles are hidden in the dashboard navigation menu and user interface, while accelerator profiles remain visible. In addition, user interface components associated with the deprecated accelerator profiles functionality are still displayed. If you enable hardware profiles, the **Hardware profiles** list is displayed instead of the **Accelerator profiles** list. To show the **Settings → Hardware profiles** option in the dashboard navigation menu, and the user interface components associated with hardware profiles, set the **disableHardwareProfiles** value to **false** in the **OdhDashboardConfig** custom resource (CR) in OpenShift. For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

- b. Optional: To change these default values, click **Customize resource requests and limit** and enter new minimum (request) and maximum (limit) values. The hardware profile specifies the number of CPUs and the amount of memory allocated to the container, setting the guaranteed minimum (request) and maximum (limit) for both.
12. Optional: In the **Model route** section, select the **Make deployed models available through an external route** checkbox to make your deployed models available to external clients.
13. To require token authentication for inference requests to the deployed model, perform the following actions:
 - a. Select **Require token authentication**.
 - b. In the **Service account name** field, enter the service account name that the token will be generated for.
 - c. To add an additional service account, click **Add a service account** and enter another service account name.
14. To specify the location of your model, select a **Connection type** that you have added. The **OCI-compliant registry**, **S3 compatible object storage**, and **URI** options are pre-installed connection types. Additional options might be available if your OpenShift AI administrator added them.
 - a. **For S3-compatible object storage:** In the **Path** field, enter the folder path that contains the model in your specified data source.



IMPORTANT

The OpenVINO Model Server runtime has specific requirements for how you specify the model path. For more information, see known issue [RHOAIENG-3025](#) in the OpenShift AI release notes.

- b. **For Open Container Image connections** In the **OCI storage location** field, enter the model URI where the model is located.

**NOTE**

If you are deploying a registered model version with an existing S3, URI, or OCI data connection, some of your connection details might be autofilled. This depends on the type of data connection and the number of matching connections available in your data science project. For example, if only one matching connection exists, fields like the path, URI, endpoint, model URI, bucket, and region might populate automatically. Matching connections will be labeled as **Recommended**.

- c. Complete the connection detail fields.
- d. Optional: If you have uploaded model files to a persistent volume claim (PVC) and the PVC is attached to your workbench, use the **Existing cluster storage** option to select the PVC and specify the path to the model file.

**IMPORTANT**

If your connection type is an S3-compatible object storage, you must provide the folder path that contains your data file. The OpenVINO Model Server runtime has specific requirements for how you specify the model path. For more information, see known issue [RHOAIENG-3025](#) in the OpenShift AI release notes.

15. (Optional) Customize the runtime parameters in the **Configuration parameters** section:
 - a. Modify the values in **Additional serving runtime arguments** to define how the deployed model behaves.
 - b. Modify the values in **Additional environment variables** to define variables in the model's environment.

The **Configuration parameters** section shows predefined serving runtime parameters, if any are available.

**NOTE**

Do not modify the port or model serving runtime arguments, because they require specific values to be set. Overwriting these parameters can cause the deployment to fail.

16. Click **Deploy**.

Verification

- Confirm that the deployed model is shown on the **Models** tab for the project, and on the **Model deployments** page of the dashboard with a checkmark in the **Status** column.

Additional resources

- [Model-serving runtimes for accelerators](#)

2.3. DEPLOYING A MODEL STORED IN AN OCI IMAGE BY USING THE CLI

You can deploy a model that is stored in an OCI image from the command line interface.

The following procedure uses the example of deploying a MobileNet v2-7 model in ONNX format, stored in an OCI image on an OpenVINO model server.



NOTE

By default in KServe, models are exposed outside the cluster and not protected with authentication.

Prerequisites

- You have stored a model in an OCI image as described in [Storing a model in an OCI image](#).
- If you want to deploy a model that is stored in a private OCI repository, you must configure an image pull secret. For more information about creating an image pull secret, see [Using image pull secrets](#).
- You are logged in to your OpenShift cluster.

Procedure

1. Create a project to deploy the model:

```
oc new-project oci-model-example
```

2. Use the OpenShift AI Applications project **kserve-ovms** template to create a **ServingRuntime** resource and configure the OpenVINO model server in the new project:

```
oc process -n redhat-ods-applications -o yaml kserve-ovms | oc apply -f -
```

3. Verify that the **ServingRuntime** named **kserve-ovms** is created:

```
oc get servingruntimes
```

The command should return output similar to the following:

NAME	DISABLED	MODELTYPE	CONTAINERS	AGE
kserve-ovms		openvino_ir	kserve-container	1m

4. Create an **InferenceService** YAML resource, depending on whether the model is stored from a private or a public OCI repository:
 - For a model stored in a public OCI repository, create an **InferenceService** YAML file with the following values, replacing **<user_name>**, **<repository_name>**, and **<tag_name>** with values specific to your environment:

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: sample-isvc-using-oci
spec:
  predictor:
    model:
```

```

    runtime: kserve-ovms # Ensure this matches the name of the ServingRuntime
resource
  modelFormat:
    name: onnx
  storageUri: oci://quay.io/<user_name>/<repository_name>:<tag_name>
  resources:
    requests:
      memory: 500Mi
      cpu: 100m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it
    limits:
      memory: 4Gi
      cpu: 500m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it

```

- For a model stored in a private OCI repository, create an **InferenceService** YAML file that specifies your pull secret in the **spec.predictor.imagePullSecrets** field, as shown in the following example:

```

apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: sample-isvc-using-private-oci
spec:
  predictor:
    model:
      runtime: kserve-ovms # Ensure this matches the name of the ServingRuntime
resource
  modelFormat:
    name: onnx
  storageUri: oci://quay.io/<user_name>/<repository_name>:<tag_name>
  resources:
    requests:
      memory: 500Mi
      cpu: 100m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it
    limits:
      memory: 4Gi
      cpu: 500m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it
    imagePullSecrets: # Specify image pull secrets to use for fetching container images,
including OCI model images
      - name: <pull-secret-name>

```

After you create the **InferenceService** resource, KServe deploys the model stored in the OCI image referred to by the **storageUri** field.

Verification

Check the status of the deployment:

```
oc get inferencervice
```

■

The command should return output that includes information, such as the URL of the deployed model and its readiness state.

2.4. DEPLOYING MODELS BY USING DISTRIBUTED INFERENCE WITH LLM-D



IMPORTANT

Distributed Inference with llm-d is currently available in Red Hat OpenShift AI 2.25 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Distributed Inference with llm-d is a Kubernetes-native, open-source framework designed for serving large language models (LLMs) at scale. You can use Distributed Inference with llm-d to simplify the deployment of generative AI, focusing on high performance and cost-effectiveness across various hardware accelerators.

Key features of Distributed Inference with llm-d include:

- Efficiently handles large models using optimizations such as prefix-cache aware routing and disaggregated serving.
- Integrates into a standard Kubernetes environment, where it leverages specialized components like the Envoy proxy to handle networking and routing, and high-performance libraries such as vLLM and NVIDIA Inference Transfer Library (NIXL).
- Tested recipes and well-known presets reduce the complexity of deploying inference at scale, so users can focus on building applications rather than managing infrastructure.

Serving models using Distributed Inference with llm-d on Red Hat OpenShift AI consists of the following steps:

1. Installing OpenShift AI.



NOTE

Because KServe Serverless conflicts with the Gateway API used for Distributed Inference with llm-d, KServe Serverless is not supported on the same cluster. Instead, use KServe RawDeployment.

2. Enabling the single model serving platform.
3. Enabling Distributed Inference with llm-d on a Kubernetes cluster.
4. Creating an LLMInferenceService Custom Resource (CR).
5. Deploying a model.

This procedure describes how to create a custom resource (CR) for an **LLMInferenceService** resource. You replace the default **InferenceService** with the **LLMInferenceService**.

Prerequisites

- You have enabled the single model-serving platform.
- You have access to an OpenShift cluster running version 4.19.9 or later.
- OpenShift Service Mesh v2 is not installed in the cluster.
- You have created a **GatewayClass** and a **Gateway** named **openshift-ai-inference** in the **openshift-ingress** namespace as described in [Gateway API with OpenShift Container Platform Networking](#).
- You have installed the **LeaderWorkerSet** Operator in OpenShift. For more information, see the OpenShift documentation.

Procedure

1. Log in to the OpenShift console as a cluster administrator.
2. Create a data science cluster initialization (DSCI) and set the **serviceMesh.managementState** to **removed**, as shown in the following example:

```
serviceMesh:
  ...
  managementState: Removed
```

3. Create a data science cluster (DSC) with the following information set in **kserve** and **serving**:

```
kserve:
  defaultDeploymentMode: RawDeployment
  managementState: Managed
  ...
serving:
  ...
  managementState: Removed
  ...
```

4. Create the **LLMInferenceService** CR with the following information:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: sample-llm-inference-service
spec:
  replicas: 2
  model:
    uri: hf://RedHatAI/Qwen3-8B-FP8-dynamic
    name: RedHatAI/Qwen3-8B-FP8-dynamic
  router:
    route: {}
    gateway: {}
    scheduler: {}
```

```

template:
  containers:
  - name: main
    resources:
      limits:
        cpu: '4'
        memory: 32Gi
        nvidia.com/gpu: "1"
      requests:
        cpu: '2'
        memory: 16Gi
        nvidia.com/gpu: "1"

```

Customize the following parameters in the **spec** section of the inference service:

- **replicas** - Specify the number of replicas.
- **model** - Provide the URI to the model based on how the model is stored (**uri**) and the model name to use in chat completion requests (**name**).
 - S3 bucket: **s3://<bucket-name>/<object-key>**
 - Persistent volume claim (PVC): **pvc://<claim-name>/<pvc-path>**
 - OCI container image: **oci://<registry_host>/<org_or_username>/<repository_name><tag_or_digest>**
 - HuggingFace: **hf://<model>/<optional-hash>**
- **router** - Provide an HTTPRoute and gateway, or leave blank to automatically create one.

5. Save the file.

2.4.1. Example usage for Distributed Inference with llm-d

These examples show how to use Distributed Inference with llm-d in common scenarios.



IMPORTANT

Distributed Inference with llm-d is currently available in Red Hat OpenShift AI 2.25 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

2.4.1.1. Single-node GPU deployment

Use single-GPU-per-replica deployment patterns for development, testing, or production deployments of smaller models, such as 7-billion-parameter models.

You can use the following examples for single-node GPU deployments:

- [Basic deployment with default scheduler](#)
- [Deployment without scheduler](#)
- [Prefill-decode separation](#)

2.4.1.2. Multi-node deployment

You can use the following examples for multi-node deployments:

- [Basic Data Parallelism \(DP\) and Expert Parallelism \(EP\) configuration](#)
- [Prefill-decode separation with a high-throughput backend](#)
- [Prefill-decode separation with a mixed backend](#)

2.4.1.3. Intelligent inference scheduler with KV cache routing

You can configure the scheduler to track key-value (KV) cache blocks across inference endpoints and route requests to the endpoint with the highest cache hit rate. This configuration improves throughput and reduces latency by maximizing cache reuse.

For an example, see [Precise Prefix KV Cache Routing](#).

2.5. MONITORING MODELS ON THE SINGLE-MODEL SERVING PLATFORM

You can monitor models that are deployed on the single-model serving platform to view performance and resource usage metrics.

2.5.1. Viewing performance metrics for a deployed model

You can monitor the following metrics for a specific model that is deployed on the single-model serving platform:

- **Number of requests** - The number of requests that have failed or succeeded for a specific model.
- **Average response time (ms)** - The average time it takes a specific model to respond to requests.
- **CPU utilization (%)** - The percentage of the CPU limit per model replica that is currently utilized by a specific model.
- **Memory utilization (%)** - The percentage of the memory limit per model replica that is utilized by a specific model.

You can specify a time range and a refresh interval for these metrics to help you determine, for example, when the peak usage hours are and how the model is performing at a specified time.

Prerequisites

- You have installed Red Hat OpenShift AI.

- A cluster admin has enabled user workload monitoring (UWM) for user-defined projects on your OpenShift cluster. For more information, see [Enabling monitoring for user-defined projects](#) and [Configuring monitoring for the single-model serving platform](#).
- You have logged in to Red Hat OpenShift AI.
- The following dashboard configuration options are set to the default values as shown:

```
disablePerformanceMetrics:false
disableKServeMetrics:false
```

For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

- You have deployed a model on the single-model serving platform by using a preinstalled runtime.



NOTE

Metrics are only supported for models deployed by using a preinstalled model-serving runtime or a custom runtime that is duplicated from a preinstalled runtime.

Procedure

1. From the OpenShift AI dashboard navigation menu, click **Data science projects**. The **Data science projects** page opens.
2. Click the name of the project that contains the data science models that you want to monitor.
3. In the project details page, click the **Models** tab.
4. Select the model that you are interested in.
5. On the **Endpoint performance** tab, set the following options:
 - **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.
 - **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed (to show the latest data). You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.
6. Scroll down to view data graphs for number of requests, average response time, CPU utilization, and memory utilization.

Verification

The **Endpoint performance** tab shows graphs of metrics for the model.

2.5.2. Viewing model-serving runtime metrics for the single-model serving platform

When a cluster administrator has configured monitoring for the single-model serving platform, non-admin users can use the OpenShift web console to view model-serving runtime metrics for the KServe component.

Prerequisites

- A cluster administrator has configured monitoring for the single-model serving platform.
- You have been assigned the **monitoring-rules-view** role. For more information, see [Granting users permission to configure monitoring for user-defined projects](#).
- You are familiar with how to monitor project metrics in the OpenShift web console. For more information, see [Monitoring your project metrics](#).

Procedure

1. Log in to the OpenShift web console.
2. Switch to the **Developer** perspective.
3. In the left menu, click **Observe**.
4. As described in [Monitoring your project metrics](#), use the web console to run queries for model-serving runtime metrics. You can also run queries for metrics that are related to OpenShift Service Mesh. Some examples are shown.
 - a. The following query displays the number of successful inference requests over a period of time for a model deployed with the vLLM runtime:

```
sum(increase(vllm:request_success_total{namespace=${namespace},model_name=${model_name}}[${rate_interval}]))
```



NOTE

Certain vLLM metrics are available only after an inference request is processed by a deployed model. To generate and view these metrics, you must first make an inference request to the model.

- b. The following query displays the number of successful inference requests over a period of time for a model deployed with the standalone TGIS runtime:

```
sum(increase(tgi_request_success{namespace=${namespace}, pod=~${model_name}-predictor-*}[${rate_interval}]))
```

- c. The following query displays the number of successful inference requests over a period of time for a model deployed with the Caikit Standalone runtime:

```
sum(increase(predict_rpc_count_total{namespace=${namespace},code=OK,model_id=${model_name}}[${rate_interval}]))
```

- d. The following query displays the number of successful inference requests over a period of time for a model deployed with the OpenVINO Model Server runtime:

```
sum(increase(ovms_requests_success{namespace=${namespace},name=${model_name}}[${rate_interval}]))
```

Additional resources

- [OVMS metrics](#)
- [TGIS metrics](#)
- [vLLM metrics](#)

CHAPTER 3. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM

You can deploy models using NVIDIA NIM inference services on the NVIDIA NIM model serving platform.

NVIDIA NIM, part of NVIDIA AI Enterprise, is a set of microservices designed for secure, reliable deployment of high performance AI model inferencing across clouds, data centers and workstations.

3.1. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM

When you have enabled the **NVIDIA NIM model serving platform**, you can start to deploy NVIDIA-optimized models on the platform.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have enabled the **NVIDIA NIM model serving platform**
- You have created a data science project.
- You have enabled support for graphic processing units (GPUs) in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).

Procedure

1. In the left menu, click **Data science projects**.
The **Data science projects** page opens.
2. Click the name of the project that you want to deploy a model in.
A project details page opens.
3. Click the **Models** tab.
4. In the **Models** section, perform one of the following actions:
 - On the **NVIDIA NIM model serving platform** tile, click **Select NVIDIA NIM** on the tile, and then click **Deploy model**.
 - If you have previously selected the NVIDIA NIM model serving type, the **Models** page displays **NVIDIA model serving enabled** on the upper-right corner, along with the **Deploy model** button. To proceed, click **Deploy model**.

The **Deploy model** dialog opens.

5. Configure properties for deploying your model as follows:
 - a. In the **Model deployment name** field, enter a unique name for the deployment.
 - b. From the **NVIDIA NIM** list, select the NVIDIA NIM model that you want to deploy. For more information, see [Supported Models](#)

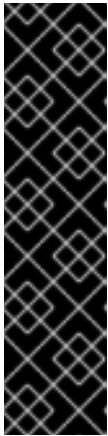
- c. In the **NVIDIA NIM storage size** field, specify the size of the cluster storage instance that will be created to store the NVIDIA NIM model.



NOTE

When resizing a PersistentVolumeClaim (PVC) backed by Amazon EBS in OpenShift AI, you may encounter **VolumeModificationRateExceeded: You've reached the maximum modification rate per volume limit**. To avoid this error, wait at least six hours between modifications per EBS volume. If you resize a PVC before the cooldown expires, the Amazon EBS CSI driver (ebs.csi.aws.com) fails with this error. This error is an Amazon EBS service limit that applies to all workloads using EBS-backed PVCs.

- d. In the **Number of model server replicas to deploy** field, specify a value.
 - e. From the **Model server size** list, select a value.
6. From the **Hardware profile** list, select a hardware profile.



IMPORTANT

By default, hardware profiles are hidden in the dashboard navigation menu and user interface, while accelerator profiles remain visible. In addition, user interface components associated with the deprecated accelerator profiles functionality are still displayed. If you enable hardware profiles, the **Hardware profiles** list is displayed instead of the **Accelerator profiles** list. To show the **Settings → Hardware profiles** option in the dashboard navigation menu, and the user interface components associated with hardware profiles, set the **disableHardwareProfiles** value to **false** in the **OdhDashboardConfig** custom resource (CR) in OpenShift. For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

7. Optional: Click **Customize resource requests and limit** and update the following values:
- a. In the **CPUs requests** field, specify the number of CPUs to use with your model server. Use the list beside this field to specify the value in cores or millicores.
 - b. In the **CPU limits** field, specify the maximum number of CPUs to use with your model server. Use the list beside this field to specify the value in cores or millicores.
 - c. In the **Memory requests** field, specify the requested memory for the model server in gibibytes (Gi).
 - d. In the **Memory limits** field, specify the maximum memory limit for the model server in gibibytes (Gi).
8. Optional: In the **Model route** section, select the **Make deployed models available through an external route** checkbox to make your deployed models available to external clients.
9. To require token authentication for inference requests to the deployed model, perform the following actions:
- a. Select **Require token authentication**.

- b. In the **Service account name** field, enter the service account name that the token will be generated for.
- c. To add an additional service account, click **Add a service account** and enter another service account name.

10. Click **Deploy**.

Verification

- Confirm that the deployed model is shown on the **Models** tab for the project, and on the **Model deployments** page of the dashboard with a checkmark in the **Status** column.

Additional resources

- [NVIDIA NIM API reference](#)
- [Supported Models](#)

3.2. VIEWING NVIDIA NIM METRICS FOR A NIM MODEL

In OpenShift AI, you can observe the following NVIDIA NIM metrics for a NIM model deployed on the NVIDIA NIM model serving platform:

- **GPU cache usage over time (ms)**
- **Current running, waiting, and max requests count**
- **Tokens count**
- **Time to first token**
- **Time per output token**
- **Request outcomes**

You can specify a time range and a refresh interval for these metrics to help you determine, for example, the peak usage hours and model performance at a specified time.

Prerequisites

- You have enabled the NVIDIA NIM model serving platform.
- You have deployed a NIM model on the NVIDIA NIM model serving platform.
- A cluster administrator has enabled metrics collection and graph generation for your deployment.
- The **disableKServeMetrics** OpenShift AI dashboard configuration option is set to its default value of **false**:

```
disableKServeMetrics: false
```

For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

Procedure

1. From the OpenShift AI dashboard navigation menu, click **Data science projects**. The **Data science projects** page opens.
2. Click the name of the project that contains the NIM model that you want to monitor.
3. In the project details page, click the **Models** tab.
4. Click the NIM model that you want to observe.
5. On the **NIM Metrics** tab, set the following options:
 - **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.
 - **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed (to show the latest data). You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.
6. Scroll down to view data graphs for NIM metrics.

Verification

The **NIM Metrics** tab shows graphs of NIM metrics for the deployed NIM model.

Additional resources

- [NVIDIA NIM observability](#)

3.3. VIEWING PERFORMANCE METRICS FOR A NIM MODEL

You can observe the following performance metrics for a NIM model deployed on the NVIDIA NIM model serving platform:

- **Number of requests** - The number of requests that have failed or succeeded for a specific model.
- **Average response time (ms)** - The average time it takes a specific model to respond to requests.
- **CPU utilization (%)** - The percentage of the CPU limit per model replica that is currently utilized by a specific model.
- **Memory utilization (%)** - The percentage of the memory limit per model replica that is utilized by a specific model.

You can specify a time range and a refresh interval for these metrics to help you determine, for example, the peak usage hours and model performance at a specified time.

Prerequisites

- You have enabled the NVIDIA NIM model serving platform.
- You have deployed a NIM model on the NVIDIA NIM model serving platform.

- A cluster administrator has enabled metrics collection and graph generation for your deployment.
- The **disableKServeMetrics** OpenShift AI dashboard configuration option is set to its default value of **false**:

```
disableKServeMetrics: false
```

For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

Procedure

1. From the OpenShift AI dashboard navigation menu, click **Data science projects**. The **Data science projects** page opens.
2. Click the name of the project that contains the NIM model that you want to monitor.
3. In the project details page, click the **Models** tab.
4. Click the NIM model that you want to observe.
5. On the **Endpoint performance** tab, set the following options:
 - **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.
 - **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed to show the latest data. You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.
6. Scroll down to view data graphs for performance metrics.

Verification

The **Endpoint performance** tab shows graphs of performance metrics for the deployed NIM model.

CHAPTER 4. DEPLOYING MODELS ON THE MULTI-MODEL SERVING PLATFORM

For deploying small and medium-sized models, OpenShift AI includes a *multi-model serving platform* that is based on the ModelMesh component. On the multi-model serving platform, multiple models can be deployed from the same model server and share the server resources.



IMPORTANT

Starting with OpenShift AI version 2.19, the multi-model serving platform based on ModelMesh is deprecated. You can continue to deploy models on the multi-model serving platform, but it is recommended that you migrate to the single-model serving platform.

For more information or for help on using the single-model serving platform, contact your account manager.

4.1. ADDING A MODEL SERVER FOR THE MULTI-MODEL SERVING PLATFORM

When you have enabled the multi-model serving platform, you must configure a model server to deploy models. If you require extra computing power for use with large datasets, you can assign accelerators to your model server.



NOTE

In OpenShift AI 2.25, Red Hat supports only NVIDIA and AMD GPU accelerators for model serving.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have created a data science project that you can add a model server to.
- You have enabled the multi-model serving platform.
- If you want to use a custom model-serving runtime for your model server, you have added and enabled the runtime. See [Adding a custom model-serving runtime](#).
- If you want to use graphics processing units (GPUs) with your model server, you have enabled GPU support in OpenShift AI. If you use NVIDIA GPUs, see [Enabling NVIDIA GPUs](#). If you use AMD GPUs, see [AMD GPU integration](#).

Procedure

1. In the left menu of the OpenShift AI dashboard, click **Data science projects**. The **Data science projects** page opens.
2. Click the name of the project that you want to configure a model server for. A project details page opens.
3. Click the **Models** tab.

4. Perform one of the following actions:

- If you see a **Multi-model serving platform** tile, click **Add model server** on the tile.
- If you do not see any tiles, click the **Add model server** button.

The **Add model server** dialog opens.

5. In the **Model server name** field, enter a unique name for the model server.

6. From the **Serving runtime** list, select a model-serving runtime that is installed and enabled in your OpenShift AI deployment.

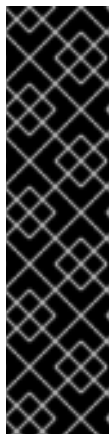


NOTE

If you are using a *custom* model-serving runtime with your model server and want to use GPUs, you must ensure that your custom runtime supports GPUs and is appropriately configured to use them.

7. In the **Number of model replicas to deploy** field, specify a value.

8. From the **Accelerator profile** list, select an accelerator profile.



IMPORTANT

By default, hardware profiles are hidden in the dashboard navigation menu and user interface, while accelerator profiles remain visible. In addition, user interface components associated with the deprecated accelerator profiles functionality are still displayed. If you enable hardware profiles, the **Hardware profiles** list is displayed instead of the **Accelerator profiles** list. To show the **Settings → Hardware profiles** option in the dashboard navigation menu, and the user interface components associated with hardware profiles, set the **disableHardwareProfiles** value to **false** in the **OdhDashboardConfig** custom resource (CR) in OpenShift. For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

9. Optional: Click **Customize resource requests and limit** and update the following values:

- In the **CPUs requests** field, specify the number of CPUs to use with your model server. Use the list beside this field to specify the value in cores or millicores.
- In the **CPU limits** field, specify the maximum number of CPUs to use with your model server. Use the list beside this field to specify the value in cores or millicores.
- In the **Memory requests** field, specify the requested memory for the model server in gibibytes (Gi).
- In the **Memory limits** field, specify the maximum memory limit for the model server in gibibytes (Gi).

10. Optional: In the **Model route** section, select the **Make deployed models available through an external route** checkbox to make your deployed models available to external clients.

11. Optional: In the **Token authentication** section, select the **Require token authentication** checkbox to require token authentication for your model server. To finish configuring token authentication, perform the following actions:

- a. In the **Service account name** field, enter a service account name for which the token will be generated. The generated token is created and displayed in the **Token secret** field when the model server is configured.
 - b. To add an additional service account, click **Add a service account** and enter another service account name.
12. Click **Add**.
 - The model server that you configured is displayed on the **Models** tab for the project, in the **Models and model servers** list.
 13. Optional: To update the model server, click the action menu () beside the model server and select **Edit model server**.

4.2. DELETING A MODEL SERVER

When you no longer need a model server to host models, you can remove it from your data science project.



NOTE

When you remove a model server, you also remove the models that are hosted on that model server. As a result, the models are no longer available to applications.

Prerequisites

- You have created a data science project and an associated model server.
- You have notified the users of the applications that access the models that the models will no longer be available.

Procedure

1. From the OpenShift AI dashboard, click **Data science projects**.
The **Data science projects** page opens.
2. Click the name of the project from which you want to delete the model server.
A project details page opens.
3. Click the **Models** tab.
4. Click the action menu () beside the project whose model server you want to delete and then click **Delete model server**.
The **Delete model server** dialog opens.
5. Enter the name of the model server in the text field to confirm that you intend to delete it.
6. Click **Delete model server**.

Verification

- The model server that you deleted is no longer displayed on the **Models** tab for the project.

4.3. DEPLOYING A MODEL BY USING THE MULTI-MODEL SERVING PLATFORM

You can deploy trained models on OpenShift AI to enable you to test and implement them into intelligent applications. Deploying a model makes it available as a service that you can access by using an API. This enables you to return predictions based on data inputs.

When you have enabled the multi-model serving platform, you can deploy models on the platform.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have enabled the multi-model serving platform.
- You have created a data science project and added a model server.
- You have access to S3-compatible object storage.
- For the model that you want to deploy, you know the associated folder path in your S3-compatible object storage bucket.

Procedure

1. In the left menu of the OpenShift AI dashboard, click **Data science projects**.
The **Data science projects** page opens.
2. Click the name of the project that you want to deploy a model in.
A project details page opens.
3. Click the **Models** tab.
4. Click **Deploy model**.
5. Configure properties for deploying your model as follows:
 - a. In the **Model name** field, enter a unique name for the model that you are deploying.
 - b. From the **Model framework** list, select a framework for your model.



NOTE

The **Model framework** list shows only the frameworks that are supported by the model-serving runtime that you specified when you configured your model server.

- c. To specify the location of the model you want to deploy from S3-compatible object storage, perform one of the following sets of actions:
 - **To use an existing connection**
 - i. Select **Existing connection**.
 - ii. From the **Name** list, select a connection that you previously defined.

- iii. In the **Path** field, enter the folder path that contains the model in your specified data source.



NOTE

If you are deploying a registered model version with an existing S3 or URI data connection, some of your connection details might be autofilled. This depends on the type of data connection and the number of matching connections available in your data science project. For example, if only one matching connection exists, fields like the path, URI, endpoint, bucket, and region might populate automatically. Matching connections will be labeled as **Recommended**.

- **To use a new connection**
 - i. To define a new connection that your model can access, select **New connection**.
 - ii. In the **Add connection** modal, select a **Connection type**. The **S3 compatible object storage** and **URI** options are pre-installed connection types. Additional options might be available if your OpenShift AI administrator added them. The **Add connection** form opens with fields specific to the connection type that you selected.
 - iii. Enter the connection detail fields.
- d. (Optional) Customize the runtime parameters in the **Configuration parameters** section:
 - i. Modify the values in **Additional serving runtime arguments** to define how the deployed model behaves.
 - ii. Modify the values in **Additional environment variables** to define variables in the model's environment.
- e. Click **Deploy**.

Verification

- Confirm that the deployed model is shown on the **Models** tab for the project, and on the **Model deployments** page of the dashboard with a checkmark in the **Status** column.

4.4. VIEWING A DEPLOYED MODEL

To analyze the results of your work, you can view a list of deployed models on Red Hat OpenShift AI. You can also view the current statuses of deployed models and their endpoints.

Prerequisites

- You have logged in to Red Hat OpenShift AI.

Procedure

1. From the OpenShift AI dashboard, click **Models → Model deployments**. The **Model deployments** page opens.

For each model, the page shows details such as the model name, the project in which the model is deployed, the model-serving runtime that the model uses, and the deployment status.

- Optional: For a given model, click the link in the **Inference endpoint** column to see the inference endpoints for the deployed model.

Verification

- A list of previously deployed data science models is displayed on the **Model deployments** page.

4.5. UPDATING THE DEPLOYMENT PROPERTIES OF A DEPLOYED MODEL

You can update the deployment properties of a model that has been deployed previously. For example, you can change the model's connection and name.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have deployed a model on OpenShift AI.

Procedure

- From the OpenShift AI dashboard, click **Models** → **Model deployments**.
The **Model deployments** page opens.
- Click the action menu () beside the model whose deployment properties you want to update and click **Edit**.
The **Edit model** dialog opens.
- Update the deployment properties of the model as follows:
 - In the **Model name** field, enter a new, unique name for your model.
 - From the **Model servers** list, select a model server for your model.
 - From the **Model framework** list, select a framework for your model.



NOTE

The **Model framework** list shows only the frameworks that are supported by the model-serving runtime that you specified when you configured your model server.

- Optionally, update the connection by specifying an existing connection or by creating a new connection.
- Click **Redeploy**.

Verification

- The model whose deployment properties you updated is displayed on the **Model deployments** page of the dashboard.

4.6. DELETING A DEPLOYED MODEL

You can delete models you have previously deployed. This enables you to remove deployed models that are no longer required.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have deployed a model.

Procedure

1. From the OpenShift AI dashboard, click **Models → Model deployments**.
The **Model deployments** page opens.
2. Click the action menu () beside the deployed model that you want to delete and click **Delete**.
The **Delete deployed model** dialog opens.
3. Enter the name of the deployed model in the text field to confirm that you intend to delete it.
4. Click **Delete deployed model**.

Verification

- The model that you deleted is no longer displayed on the **Model deployments** page.

4.7. CONFIGURING MONITORING FOR THE MULTI-MODEL SERVING PLATFORM

The multi-model serving platform includes model and model server metrics for the ModelMesh component. ModelMesh generates its own set of metrics and does not rely on the underlying model-serving runtimes to provide them. The set of metrics that ModelMesh generates includes metrics for model request rates and timings, model loading and unloading rates, times and sizes, internal queuing delays, capacity and usage, cache state, and least recently-used models. For more information, see [ModelMesh metrics](#).

After you have configured monitoring, you can view metrics for the ModelMesh component.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You are familiar with [creating a config map](#) for monitoring a user-defined workflow. You will perform similar steps in this procedure.
- You are familiar with [enabling monitoring](#) for user-defined projects in OpenShift. You will perform similar steps in this procedure.

- You have [assigned](#) the **monitoring-rules-view** role to users that will monitor metrics.

Procedure

1. In a terminal window, if you are not already logged in to your OpenShift cluster as a cluster administrator, log in to the OpenShift CLI (**oc**) as shown in the following example:

```
$ oc login <openshift_cluster_url> -u <admin_username> -p <password>
```

2. Define a **ConfigMap** object in a YAML file called **uwm-cm-conf.yaml** with the following contents:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: user-workload-monitoring-config
  namespace: openshift-user-workload-monitoring
data:
  config.yaml: |
    prometheus:
      logLevel: debug
      retention: 15d
```

The **user-workload-monitoring-config** object configures the components that monitor user-defined projects. Observe that the retention time is set to the recommended value of 15 days.

3. Apply the configuration to create the **user-workload-monitoring-config** object.

```
$ oc apply -f uwm-cm-conf.yaml
```

4. Define another **ConfigMap** object in a YAML file called **uwm-cm-enable.yaml** with the following contents:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: cluster-monitoring-config
  namespace: openshift-monitoring
data:
  config.yaml: |
    enableUserWorkload: true
```

The **cluster-monitoring-config** object enables monitoring for user-defined projects.

5. Apply the configuration to create the **cluster-monitoring-config** object.

```
$ oc apply -f uwm-cm-enable.yaml
```

4.8. VIEWING MODEL-SERVING RUNTIME METRICS FOR THE MULTI-MODEL SERVING PLATFORM

After a cluster administrator has configured monitoring for the multi-model serving platform, non-admin users can use the OpenShift web console to view model-serving runtime metrics for the ModelMesh component.

Prerequisites

- A cluster administrator has configured monitoring for the multi-model serving platform.
- You have been assigned the **monitoring-rules-view** role. For more information, see [Granting users permission to configure monitoring for user-defined projects](#).
- You are familiar with how to monitor project metrics in the OpenShift web console. For more information, see [Monitoring your project metrics](#).

Procedure

1. Log in to the OpenShift web console.
2. Switch to the **Developer** perspective.
3. In the left menu, click **Observe**.
4. As described in [Monitoring your project metrics](#), use the web console to run queries for **modelmesh_*** metrics.

4.9. VIEWING PERFORMANCE METRICS FOR ALL MODELS ON A MODEL SERVER

You can monitor the following metrics for all the models that are deployed on a model server:

- **HTTP requests per 5 minutes** - The number of HTTP requests that have failed or succeeded for all models on the server.
- **Average response time (ms)** - For all models on the server, the average time it takes the model server to respond to requests.
- **CPU utilization (%)** - The percentage of the CPU's capacity that is currently being used by all models on the server.
- **Memory utilization (%)** - The percentage of the system's memory that is currently being used by all models on the server.

You can specify a time range and a refresh interval for these metrics to help you determine, for example, when the peak usage hours are and how the models are performing at a specified time.

Prerequisites

- You have installed Red Hat OpenShift AI.
- On the OpenShift cluster where OpenShift AI is installed, user workload monitoring is enabled.
- You have logged in to Red Hat OpenShift AI.
- You have deployed models on the multi-model serving platform.

Procedure

1. From the OpenShift AI dashboard navigation menu, click **Data science projects**.
The **Data science projects** page opens.
2. Click the name of the project that contains the data science models that you want to monitor.
3. In the project details page, click the **Models** tab.
4. In the row for the model server that you are interested in, click the action menu (⋮) and then select **View model server metrics**.
5. Optional: On the metrics page for the model server, set the following options:
 - **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.
 - **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed (to show the latest data). You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.
6. Scroll down to view data graphs for HTTP requests per 5 minutes, average response time, CPU utilization, and memory utilization.

Verification

On the metrics page for the model server, the graphs provide data on performance metrics.

4.10. VIEWING HTTP REQUEST METRICS FOR A DEPLOYED MODEL

You can view a graph that illustrates the HTTP requests that have failed or succeeded for a specific model that is deployed on the multi-model serving platform.

Prerequisites

- You have installed Red Hat OpenShift AI.
- On the OpenShift cluster where OpenShift AI is installed, user workload monitoring is enabled.
- The following dashboard configuration options are set to the default values as shown:

```
disablePerformanceMetrics:false  
disableKServeMetrics:false
```

For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

- You have logged in to Red Hat OpenShift AI.
- You have deployed models on the multi-model serving platform.

Procedure

1. From the OpenShift AI dashboard, click **Models → Model deployments**.
2. On the **Model deployments** page, select the model that you are interested in.

3. Optional: On the **Endpoint performance** tab, set the following options:

- **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.
- **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed (to show the latest data). You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.

Verification

The **Endpoint performance** tab shows a graph of the HTTP metrics for the model.

CHAPTER 5. MAKING INFERENCE REQUESTS TO DEPLOYED MODELS

When you deploy a model, it is available as a service that you can access with API requests. This allows you to get predictions from your model based on the data you provide in the request.

5.1. ACCESSING THE AUTHENTICATION TOKEN FOR A DEPLOYED MODEL

If you secured your model inference endpoint by enabling token authentication, you must know how to access your authentication token so that you can specify it in your inference requests.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have deployed a model by using the single-model serving platform.

Procedure

1. From the OpenShift AI dashboard, click **Data science projects**.
The **Data science projects** page opens.
2. Click the name of the project that contains your deployed model.
A project details page opens.
3. Click the **Models** tab.
4. In the **Models and model servers** list, expand the section for your model.
Your authentication token is shown in the **Token authentication** section, in the **Token secret** field.
5. Optional: To copy the authentication token for use in an inference request, click the **Copy** button () next to the token value.

5.2. ACCESSING THE INFERENCE ENDPOINT FOR A DEPLOYED MODEL

To make inference requests to your deployed model, you must know how to access the inference endpoint that is available.

For a list of paths to use with the supported runtimes and example commands, see [Inference endpoints](#).

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have deployed a model by using the single-model serving platform.
- If you enabled token authentication for your deployed model, you have the associated token value.

Procedure

1. From the OpenShift AI dashboard, click **Models → Model deployments**.
The inference endpoint for the model is shown in the **Inference endpoint** field.
2. Depending on what action you want to perform with the model (and if the model supports that action), copy the inference endpoint and then add a path to the end of the URL.
3. Use the endpoint to make API requests to your deployed model.

Additional resources

- [Text Generation Inference Server \(TGIS\)](#)
- [Caikit API documentation](#)
- [Caikit NLP GitHub project](#)
- [OpenVINO KServe-compatible REST API documentation](#)
- [OpenAI API documentation](#)

5.3. MAKING INFERENCE REQUESTS TO MODELS DEPLOYED ON THE SINGLE-MODEL SERVING PLATFORM

When you deploy a model by using the single-model serving platform, the model is available as a service that you can access using API requests. This enables you to return predictions based on data inputs. To use API requests to interact with your deployed model, you must know the inference endpoint for the model.

In addition, if you secured your inference endpoint by enabling token authentication, you must know how to access your authentication token so that you can specify this in your inference requests.

5.4. INFERENCE ENDPOINTS

These examples show how to use inference endpoints to query the model.



NOTE

If you enabled token authentication when deploying the model, add the **Authorization** header and specify a token value.

5.4.1. Caikit TGIS ServingRuntime for KServe

- **:443/api/v1/task/text-generation**
- **:443/api/v1/task/server-streaming-text-generation**

Example command

```
curl --json '{"model_id": "<model_name__>", "inputs": "<text>"}'
https://<inference_endpoint_url>:443/api/v1/task/server-streaming-text-generation -H 'Authorization:
Bearer <token>'
```

5.4.2. Caikit Standalone ServingRuntime for KServe

If you are serving multiple models, you can query `/info/models` or `:443 caikit.runtime.info.InfoService/GetModelsInfo` to view a list of served models.

REST endpoints

- `/api/v1/task/embedding`
- `/api/v1/task/embedding-tasks`
- `/api/v1/task/sentence-similarity`
- `/api/v1/task/sentence-similarity-tasks`
- `/api/v1/task/rerank`
- `/api/v1/task/rerank-tasks`
- `/info/models`
- `/info/version`
- `/info/runtime`

gRPC endpoints

- `:443 caikit.runtime.Nlp.NlpService/EmbeddingTaskPredict`
- `:443 caikit.runtime.Nlp.NlpService/EmbeddingTasksPredict`
- `:443 caikit.runtime.Nlp.NlpService/SentenceSimilarityTaskPredict`
- `:443 caikit.runtime.Nlp.NlpService/SentenceSimilarityTasksPredict`
- `:443 caikit.runtime.Nlp.NlpService/RerankTaskPredict`
- `:443 caikit.runtime.Nlp.NlpService/RerankTasksPredict`
- `:443 caikit.runtime.info.InfoService/GetModelsInfo`
- `:443 caikit.runtime.info.InfoService/GetRuntimeInfo`



NOTE

By default, the Caikit Standalone Runtime exposes REST endpoints. To use gRPC protocol, manually deploy a custom Caikit Standalone ServingRuntime. For more information, see [Adding a custom model-serving runtime for the single-model serving platform](#).

An example manifest is available in the [caikit-tgis-serving GitHub repository](#).

Example command

REST

```
curl -H 'Content-Type: application/json' -d '{"inputs": "<text>", "model_id": "<model_id>"}'
<inference_endpoint_url>/api/v1/task/embedding -H 'Authorization: Bearer <token>'
```

gRPC

```
grpcurl -d '{"text": "<text>"}' -H \"mm-model-id: <model_id>\" <inference_endpoint_url>:443
caikit.runtime.Nlp.NlpService/EmbeddingTaskPredict -H 'Authorization: Bearer <token>'
```

5.4.3. TGIS Standalone ServingRuntime for KServe



IMPORTANT

The **Text Generation Inference Server (TGIS) Standalone ServingRuntime** for KServe is deprecated. For more information, see [OpenShift AI release notes](#).

- **:443 fmaas.GenerationService/Generate**
- **:443 fmaas.GenerationService/GenerateStream**



NOTE

To query the endpoint for the TGIS standalone runtime, you must also download the files in the [proto](#) directory of the OpenShift AI **text-generation-inference** repository.

Example command

```
grpcurl -proto text-generation-inference/proto/generation.proto -d '{"requests": [{"text": "<text>"}]}' -H
'Authorization: Bearer <token>' -insecure <inference_endpoint_url>:443
fmaas.GenerationService/Generate
```

5.4.4. OpenVINO Model Server

- **/v2/models/<model-name>/infer**

Example command

```
curl -ks <inference_endpoint_url>/v2/models/<model_name>/infer -d '{ "model_name": "
<model_name>", "inputs": [{ "name": "<name_of_model_input>", "shape": [<shape>], "datatype": "
<data_type>", "data": [<data>] } ]}' -H 'Authorization: Bearer <token>'
```

5.4.5. vLLM NVIDIA GPU ServingRuntime for KServe

- **:443/version**
- **:443/docs**
- **:443/v1/models**
- **:443/v1/chat/completions**

- **:443/v1/completions**
- **:443/v1/embeddings**
- **:443/tokenize**
- **:443/detokenize**



NOTE

- The vLLM runtime is compatible with the OpenAI REST API. For a list of models that the vLLM runtime supports, see [Supported models](#).
- To use the embeddings inference endpoint in vLLM, you must use an embeddings model that the vLLM supports. You cannot use the embeddings endpoint with generative models. For more information, see [Supported embeddings models in vLLM](#).
- As of vLLM v0.5.5, you must provide a chat template while querying a model using the **/v1/chat/completions** endpoint. If your model does not include a predefined chat template, you can use the **chat-template** command-line parameter to specify a chat template in your custom vLLM runtime, as shown in the example. Replace **<CHAT_TEMPLATE>** with the path to your template.

containers:

- args:

- --chat-template=<CHAT_TEMPLATE>

You can use the chat templates that are available as **.jinja** files [here](#) or with the vLLM image under **/app/data/template**. For more information, see [Chat templates](#).

As indicated by the paths shown, the single-model serving platform uses the HTTPS port of your OpenShift router (usually port 443) to serve external API requests.

Example command

```
curl -v https://<inference_endpoint_url>:443/v1/chat/completions -H "Content-Type: application/json" -d '{ "messages": [ { "role": "<role>", "content": "<content>" } ] }' -H 'Authorization: Bearer <token>'
```

5.4.6. vLLM Intel Gaudi Accelerator ServingRuntime for KServe

See [vLLM NVIDIA GPU ServingRuntime for KServe](#).

5.4.7. vLLM AMD GPU ServingRuntime for KServe

See [vLLM NVIDIA GPU ServingRuntime for KServe](#).

5.4.8. vLLM Spyre AI Accelerator ServingRuntime for KServe



IMPORTANT

Support for IBM Spyre AI Accelerators on x86 is currently available in Red Hat OpenShift AI 2.25 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

See [vLLM NVIDIA GPU ServingRuntime for KServe](#).

5.4.9. NVIDIA Triton Inference Server

REST endpoints

- **v2/models/[versions/<model_version>]/infer**
- **v2/models/<model_name>[/versions/<model_version>]**
- **v2/health/ready**
- **v2/health/live**
- **v2/models/<model_name>[/versions/]/ready**
- **v2**



NOTE

ModelMesh does not support the following REST endpoints:

- **v2/health/live**
- **v2/health/ready**
- **v2/models/<model_name>[/versions/]/ready**

Example command

```
curl -ks <inference_endpoint_url>/v2/models/<model_name>/infer -d '{ "model_name": "  
<model_name>", "inputs": [{ "name": "<name_of_model_input>", "shape": [<shape>], "datatype": "  
<data_type>", "data": [<data> ]}]}' -H 'Authorization: Bearer <token>'
```

gRPC endpoints

- **:443 inference.GRPCInferenceService/ModelInfer**
- **:443 inference.GRPCInferenceService/ModelReady**
- **:443 inference.GRPCInferenceService/ModelMetadata**

- **:443 inference.GRPCInferenceService/ServerReady**
- **:443 inference.GRPCInferenceService/ServerLive**
- **:443 inference.GRPCInferenceService/ServerMetadata**

Example command

```
grpcurl -cacert ./openshift_ca_istio_knative.crt -proto ./grpc_predict_v2.proto -d @ -H "Authorization: Bearer <token>" <inference_endpoint_url>:443 inference.GRPCInferenceService/ModelMetadata
```

5.4.10. Seldon MLServer

REST endpoints

- **v2/models/[versions/<model_version>]/infer**
- **v2/models/<model_name>[/versions/<model_version>]**
- **v2/health/ready**
- **v2/health/live**
- **v2/models/<model_name>[/versions/]ready**
- **v2**

Example command

```
curl -ks <inference_endpoint_url>/v2/models/<model_name>/infer -d '{ "model_name": "<model_name>", "inputs": [{ "name": "<name_of_model_input>", "shape": [<shape>], "datatype": "<data_type>", "data": [<data>] } ]}' -H 'Authorization: Bearer <token>'
```

gRPC endpoints

- **:443 inference.GRPCInferenceService/ModelInfer**
- **:443 inference.GRPCInferenceService/ModelReady**
- **:443 inference.GRPCInferenceService/ModelMetadata**
- **:443 inference.GRPCInferenceService/ServerReady**
- **:443 inference.GRPCInferenceService/ServerLive**
- **:443 inference.GRPCInferenceService/ServerMetadata**

Example command

```
grpcurl -cacert ./openshift_ca_istio_knative.crt -proto ./grpc_predict_v2.proto -d @ -H "Authorization: Bearer <token>" <inference_endpoint_url>:443 inference.GRPCInferenceService/ModelMetadata
```


5.4.11. Additional resources

- [Text Generation Inference Server \(TGIS\)](#)
- [Caikit API documentation](#)
- [Caikit NLP GitHub project](#)
- [OpenVINO KServe-compatible REST API documentation](#)
- [OpenAI API documentation](#)
- [Open Inference Protocol](#)
- [Supported model-serving runtimes.](#)